

UML-диаграммы

Ищенко Алексей Петрович

Диаграмма последовательностей (sequence diagram)

- В прошлый раз рассматривалась диаграмма объектов, которая показывает отношения между объектами в некоторый момент времени, т. е. предоставляет нам снимок состояния системы, являясь статической. **Диаграмма же последовательностей отображает взаимодействие объектов в динамике.**
- В UML взаимодействие объектов понимается как обмен информацией между ними. При этом информация принимает вид сообщений. Кроме того, *что сообщение несет какую-то информацию, оно некоторым образом также влияет на получателя.* Как видим, в этом плане UML полностью соответствует основным принципам ООП, в соответствии с которыми информационное взаимодействие между объектами сводится к отправке и приему сообщений.

Диаграмма последовательностей (sequence diagram)

- *Диаграмма последовательностей* относится к диаграммам взаимодействия UML, описывающим поведенческие аспекты системы, но *рассматривает взаимодействие объектов во времени*. Другими словами, *диаграмма последовательностей отображает временные особенности передачи и приема сообщений объектами*.
- Можно сказать, что нечто подобное делает и *диаграмма прецедентов*. Да, действительно, *диаграммы последовательностей можно (и нужно!) использовать для уточнения диаграмм прецедентов*, более детального описания логики сценариев использования. Это отличное средство документирования проекта с точки зрения сценариев использования! *Диаграммы последовательностей обычно содержат объекты, которые взаимодействуют в рамках сценария, сообщения, которыми они обмениваются, и возвращаемые результаты, связанные с сообщениями*. Впрочем, часто возвращаемые результаты обозначают лишь в том случае, если это не очевидно из контекста.

Диаграмма последовательностей (sequence diagram)

Теперь о том, какие обозначения используются на диаграмме последовательностей. Как и ранее, объекты обозначаются прямоугольниками с подчеркнутыми именами (чтобы отличить их от классов), сообщения (вызовы методов) - линиями со стрелками, возвращаемые результаты - пунктирными линиями со стрелками. Прямоугольники на вертикальных линиях под каждым из объектов показывают "время жизни" (фокус) объектов. Впрочем, довольно часто их не изображают на диаграмме, все это зависит от индивидуального стиля проектирования.

Диаграмма последовательностей (sequence diagram)

Смысл диаграммы вполне понятен: студент хочет записаться на некий семинар, предлагаемый в рамках некоторого учебного курса. С этой целью проводится проверка подготовленности студента, для чего запрашивается список (история) семинаров курса, уже пройденных студентом (перейти к следующему семинару можно, лишь проработав материал предыдущих занятий - знакомая картина, не правда ли?). После получения истории семинаров объект класса "Слушатель" получает статус подготовленности, на основе которой студенту сообщается результат (статус) его попытки записи на семинар. Кстати, обратите внимание на вызов методов. Как видите, все просто!

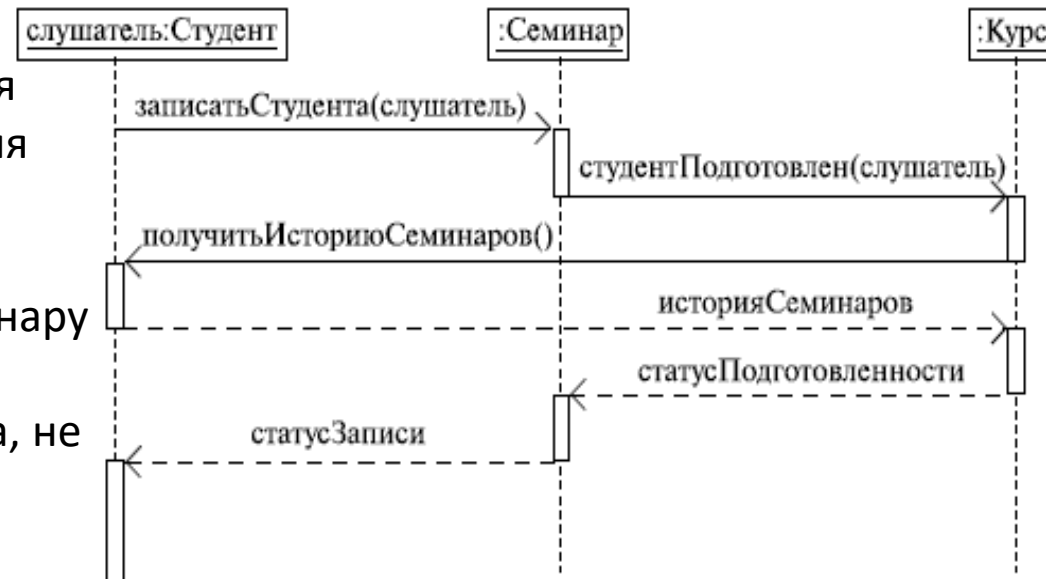


Диаграмма последовательностей (sequence diagram)

- А вот что описывает следующая диаграмма - работа обычного домового лифта, которым мы пользуемся каждый день! Кстати, посмотрите на имена объектов - видно, что это уже несколько иной стиль проектирования, чем в предыдущем примере.

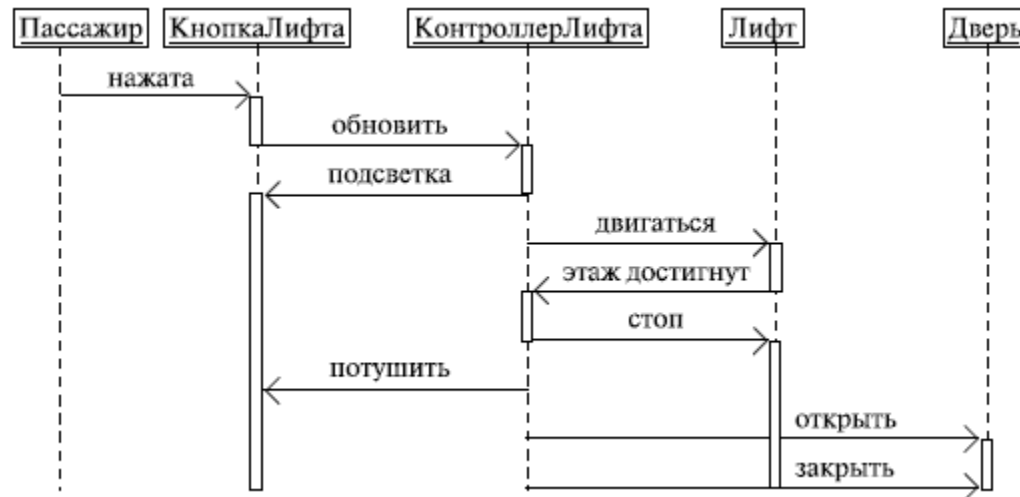


Диаграмма последовательностей (sequence diagram)

- И наконец, *еще один пример*: мобильный телефон.

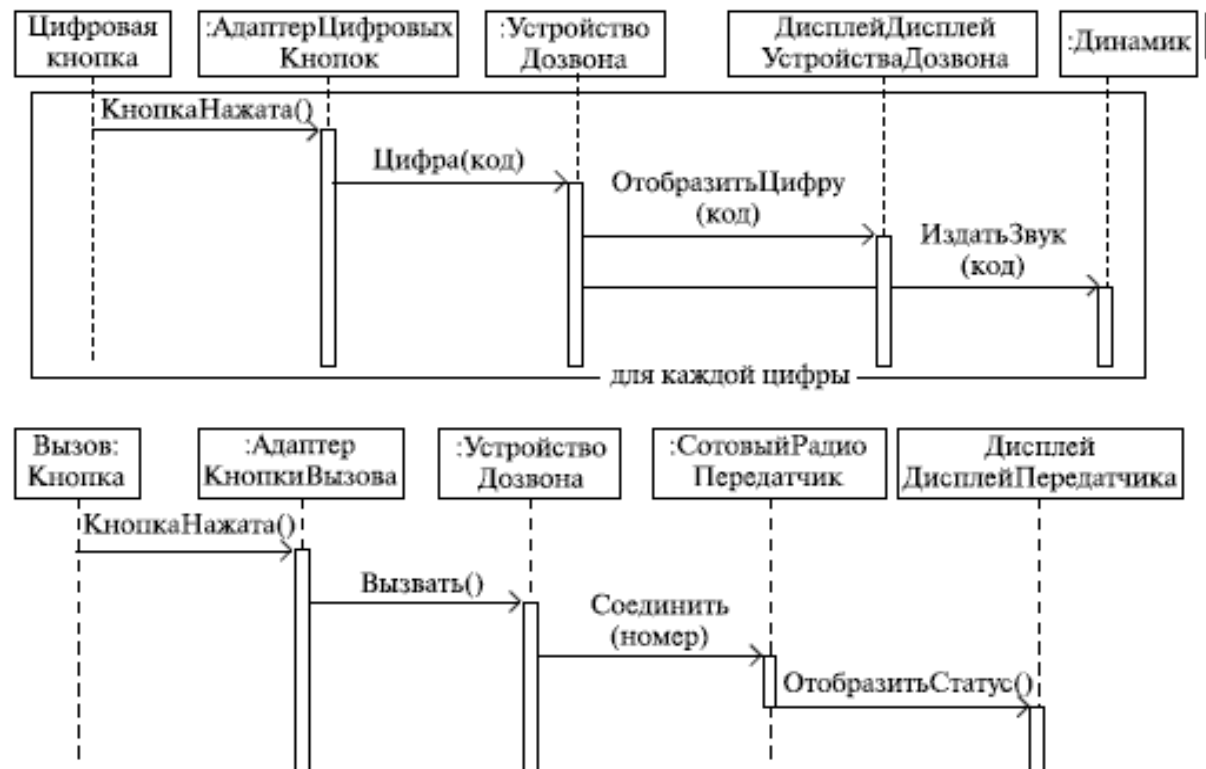


Диаграмма взаимодействия (кооперации, collaboration diagram)

Диаграммы последовательностей - это отличное средство документирования поведения системы, детализации логики сценариев использования; но есть еще один способ - использовать диаграммы взаимодействия. Диаграмма взаимодействия *показывает поток сообщений между объектами системы и основные ассоциации между ними* и по сути, как уже было сказано выше, является альтернативой диаграммы последовательностей. Внимательный читатель, возможно, скажет, что *диаграмма объектов* делает то же самое, - и будет не прав. *Диаграмма объектов показывает статику*, некий снимок системы, связи между объектами в данный момент времени, диаграмма же взаимодействия, как и диаграмма последовательностей, показывает взаимодействие (извините за невольный каламбур) объектов во времени, т. е. в динамике.

Диаграмма взаимодействия (кооперации, collaboration diagram)

- Следует отметить, что использование диаграммы последовательностей или диаграммы взаимодействия - личный выбор каждого проектировщика и зависит от индивидуального стиля проектирования. На обозначениях, применяемых на диаграмме взаимодействия, думаем, не стоит останавливаться подробно. Здесь все стандартно: объекты обозначаются прямоугольниками с подчеркнутыми именами (чтобы отличить их от классов), ассоциации между объектами указываются в виде соединяющих их линий, над ними может быть изображена стрелка с указанием названия сообщения и его порядкового номера.
- Необходимость номера сообщения объясняется очень просто - в отличие от диаграммы последовательностей, *время на диаграмме взаимодействия не показывается в виде отдельного измерения*. Поэтому последовательность передачи сообщений можно указать только с помощью их нумерации. В этом и состоит вероятная причина пренебрежения этим видом диаграмм многими проектировщиками.

Диаграмма взаимодействия (кооперации, collaboration diagram)

- эта диаграмма описывает (очень грубо) работу персонала библиотеки по обслуживанию клиентов: библиотекарь получает заказ от клиента, поручает сотруднику найти информацию по нужной клиенту книге, а после получения данных поручает еще одному сотруднику выдать книгу клиенту.



Как видите, эта диаграмма описывает (очень грубо) работу персонала библиотеки по обслуживанию клиентов: библиотекарь получает заказ от клиента, поручает сотруднику найти информацию по нужной клиенту книге, а после получения данных поручает еще одному сотруднику выдать книгу клиенту.

Диаграмма взаимодействия (кооперации, collaboration diagram)

Диаграмма описывает процесс управления учебными курсами (очевидно, путем создания их из готовых модулей) для некоего учебного центра.



Диаграмма взаимодействия (кооперации, collaboration diagram)

Это последний пример - мобильный телефон! Как видим, это просто другая форма представления, к тому же менее удобная. Впрочем, в команде могут работать различные люди, с различными предпочтениями и особенностями восприятия, так что какой вид диаграмм использовать для описания логики сценариев - диаграммы последовательностей или диаграммы кооперации - решать вам.



Диаграмма состояний (statechart diagram)

- *Объекты характеризуются поведением и состоянием, в котором находятся. Например, человек может быть новорожденным, младенцем, ребенком, подростком или взрослым. Другими словами, объекты что-то делают и что-то "знают".* **Диаграммы состояний** применяются для того, чтобы объяснить, каким образом работают сложные объекты. Несмотря на то что смысл понятия "состояние" интуитивно понятен, все же приведем его определение в таком виде, в каком его дают классики и Zicom Mentor:
- **Состояние (state)** - ситуация в жизненном цикле объекта, во время которой он удовлетворяет некоторому условию, выполняет определенную деятельность или ожидает какого-то события. Состояние объекта определяется значениями некоторых его атрибутов и присутствием или отсутствием связей с другими объектами.

Диаграмма состояний (statechart diagram)

- Диаграмма состояний показывает, как объект переходит из одного состояния в другое. Очевидно, что диаграммы состояний служат для моделирования динамических аспектов системы (как и диаграммы последовательностей, кооперации, прецедентов и, как мы увидим далее, диаграммы деятельности). Часто можно услышать, что *диаграмма состояний показывает автомат*. Диаграмма состояний полезна при *моделировании жизненного цикла* объекта (как и ее частная разновидность - диаграмма деятельности, о которой мы будем говорить далее).
- От других диаграмм диаграмма состояний отличается тем, что описывает процесс изменения состояний только одного экземпляра определенного класса - одного объекта, причем объекта *реактивного*, то есть объекта, поведение которого характеризуется его реакцией на внешние события. Понятие жизненного цикла применимо как раз к реактивным объектам, настоящее состояние (и поведение) которых обусловлено их прошлым состоянием. Но диаграммы состояний важны не только для описания динамики отдельного объекта. Они могут использоваться для *конструирования исполняемых систем* путем прямого и обратного проектирования.

Диаграмма состояний (statechart diagram)

- Поговорим об обозначениях на диаграммах состояний. Скругленные прямоугольники представляют состояния, через которые проходит объект в течение своего жизненного цикла. Стрелками показываются переходы между состояниями, которые вызваны выполнением методов описываемого диаграммой объекта. Существует также *два вида псевдосостояний*: *начальное*, в котором находится объект сразу после его создания (обозначается сплошным кружком), и *конечное*, которое объект не может покинуть, если перешел в него (обозначается кружком, обведенным окружностью).
- Приведем пример простейшей диаграммы состояний:

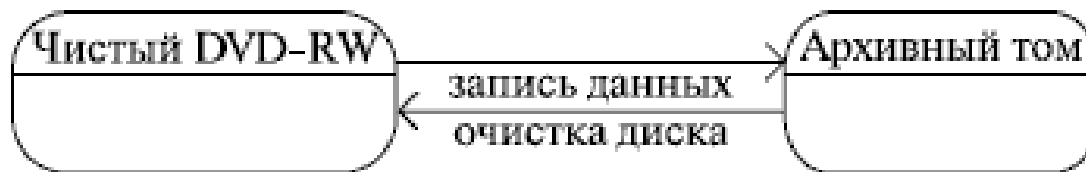


Диаграмма состояний (statechart diagram)

- Здесь мы видим *составное состояние*, включающее другие состояния, одно из которых содержит также параллельные *подсостояния*. Мы не говорили ранее о том, как все это обозначается, но ведь и так понятно - это диаграмма прохождения академического курса студентом. Для того чтобы пройти курс, студент должен выполнить лабораторные работы, защитить курсовой проект и сдать экзамен. Все просто!

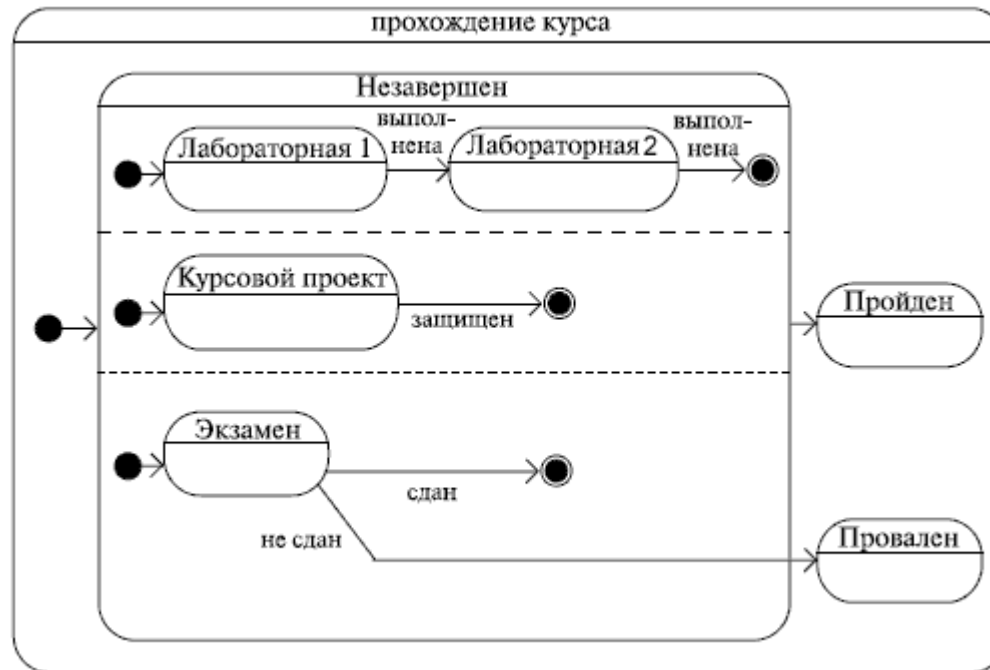
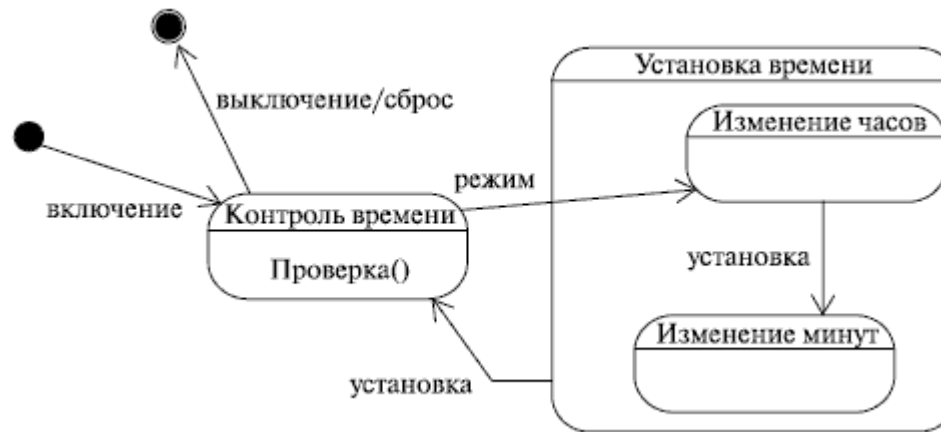


Диаграмма состояний (statechart diagram)



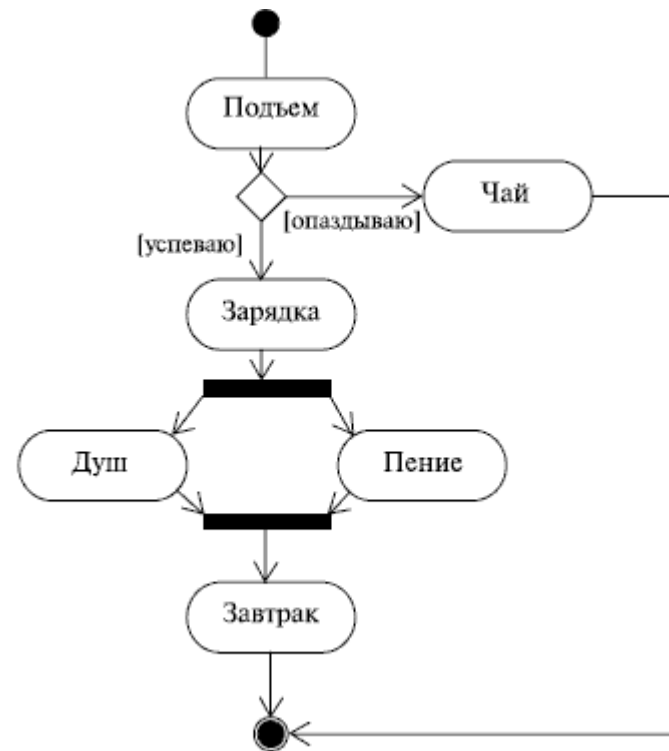
- Это устройство - таймер! Такой прибор может применяться в составе различных реле, например, для отключения телевизора по истечении указанного промежутка времени. Основное его назначение - контроль времени (проверка, не истек ли указанный промежуток), но у него есть еще один режим работы - установка. По истечении указанного промежутка времени или при "сбросе" устройство отключается.

Диаграмма активности (деятельности, activity diagram)

- Моделируя поведение проектируемой системы, часто недостаточно изобразить процесс смены ее состояний, а нужно также раскрыть детали алгоритмической реализации операций, выполняемых системой. Как мы уже говорили, для этой цели традиционно использовались блок-схемы или структурные схемы алгоритмов. В UML для этого существуют **диаграммы деятельности**, являющиеся частным случаем диаграмм состояний. Диаграммы деятельности удобно применять для визуализации алгоритмов, по которым работают операции классов.
- **Алгоритм** - последовательность определенных действий или элементарных операций, выполнение которых приводит к получению желаемого результата.
- Алгоритмы окружают нас повсюду, хоть мы и редко задумываемся об этом. Вспомните кулинарные рецепты или руководства по эксплуатации бытовых приборов! Конечно, отечественный потребитель привык жить по принципу "если ничего не помогает, прочтите, наконец, инструкцию", но факт остается фактом: чем сложнее устройство или система, тем важнее строго следовать алгоритму.

Диаграмма активности (деятельности, activity diagram)

Обозначения на диаграмме активности также напоминают те, которые мы встречали на блок-схеме, хотя есть, как мы увидим далее, и некоторые существенные отличия. С другой стороны, нотация диаграмм активности очень похожа на ту, которая используется в диаграммах состояний. Но, наверное, лучше будет просто показать пример:



Многие из нас именно так начинают свой день, не правда ли? Обратите внимание на то, как изображено параллельное пение и принятие душа, - на обычной блок-схеме это было бы невозможно!

Диаграмма активности (деятельности, activity diagram)

- оформление заказа в интернет-магазине!



Диаграмма активности (деятельности, activity diagram)

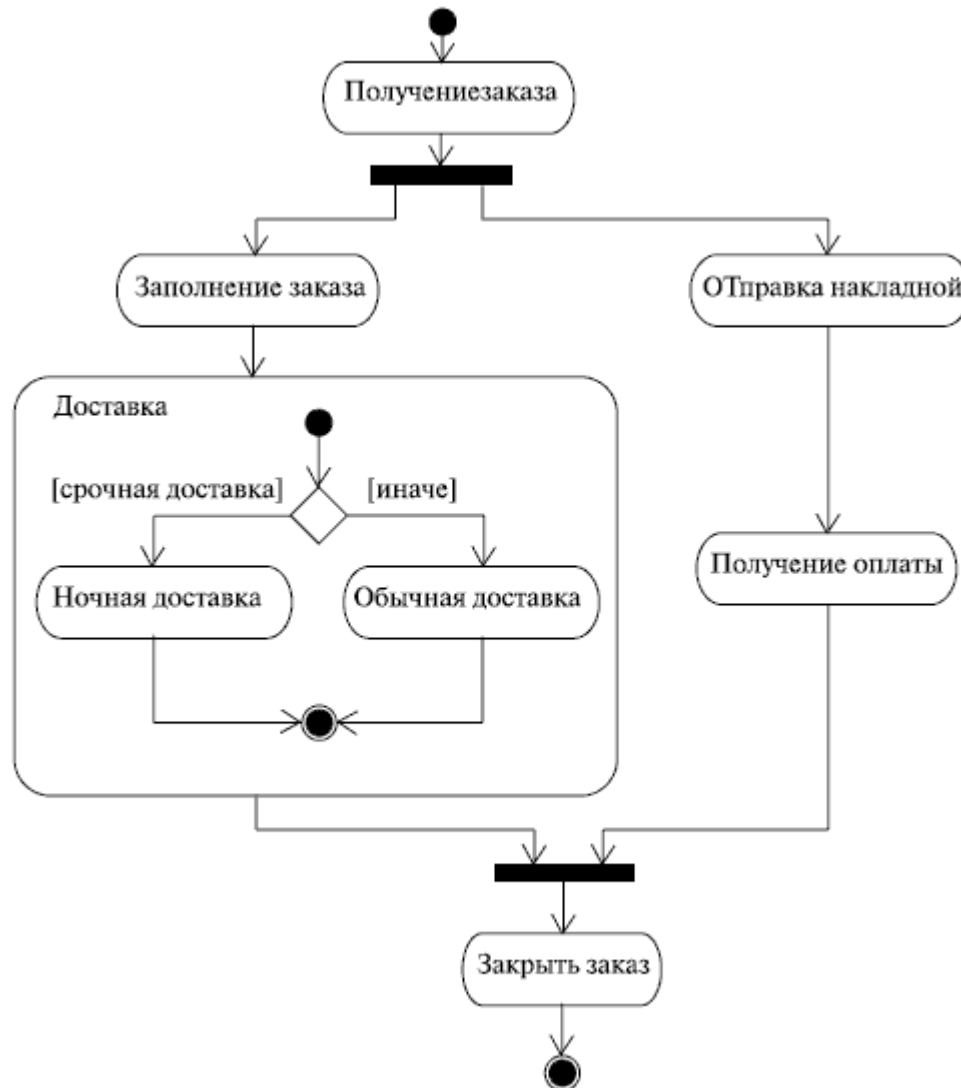


Диаграмма развертывания (deployment diagram)

- Когда мы пишем программу, мы пишем ее для того, чтобы запускать на компьютере, который имеет некоторую аппаратную конфигурацию и работает под управлением некоторой операционной системы. Корпоративные приложения часто требуют для своей работы некоторой *ИТ-инфраструктуры*, хранят информацию в базах данных, расположенных где-то на серверах компании, вызывают веб-сервисы, используют общие ресурсы и т. д. В таких случаях неплохо бы иметь *графическое представление инфраструктуры, на которую будет развернуто приложение*. Вот для этого-то и нужны **диаграммы развертывания**, которые иногда называют диаграммами размещения.
- Какую пользу можно извлечь из диаграмм развертывания? Во-первых, графическое представление ИТ-инфраструктуры может помочь *более рационально распределить компоненты системы по узлам сети*, от чего, как известно, зависит в том числе и производительность системы. Во-вторых, такая диаграмма может помочь *решить множество вспомогательных задач*, связанных, например, с обеспечением безопасности.

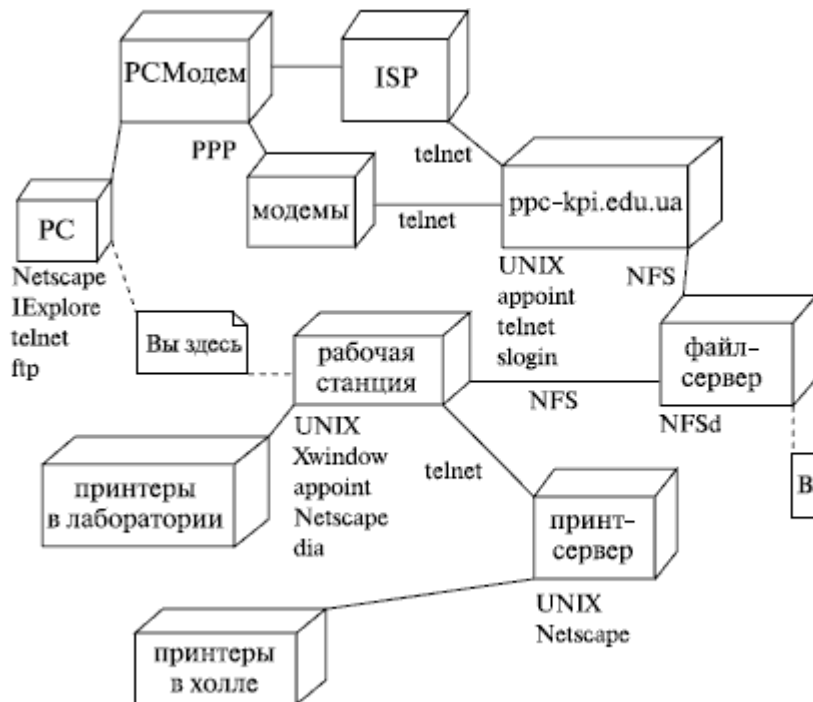
Диаграмма развертывания (deployment diagram)

- *Диаграмма развертывания* показывает топологию системы и распределение компонентов системы по ее узлам, а также соединения - маршруты передачи информации между аппаратными узлами. Это единственная диаграмма, на которой применяются "трехмерные" обозначения: узлы системы обозначаются кубиками. Все остальные обозначения в UML - плоские фигуры.



Диаграмма развертывания (deployment diagram)

- А вот *диаграмма развертывания* с большим количеством узлов



Это инфраструктура некоего учебного заведения, включающая шлюз, файл-сервер, принт-сервер, принтеры в лабораториях и холле и т. д. Пользователь (вероятно, студент или преподаватель) может получить доступ к этим ресурсам либо со своей домашней машины, либо с рабочих станций, находящихся в лабораториях вуза. Обратите внимание на подписи под линиями, показывающими линии передачи информации, например, видно, что рабочая станция получает доступ к файлам, хранящимся на файл-сервере, посредством NFS. Также хорошая идея - рядом с обозначением узла перечислить программное обеспечение, установленное на данном узле, как это сделано, например, для рабочей станции.

ООП и последовательность построения диаграмм

Естественно, *читать об UML недостаточно - надо им пользоваться!* Может быть, даже сразу вы чего-то и не поймете, но по мере увеличения опыта использования *UML* вы все лучше начнете понимать его конструкции. Так же как и другие языки, *UML* требует особого способа мышления, умения рассматривать систему с разных сторон и точек зрения.

Можно дать множество рекомендаций относительно того, какие же именно диаграммы строить и как. Прежде всего, вы должны ответить для себя на такие вопросы:

- Какие именно виды диаграмм лучше всего отражают архитектуру системы и возможные технические риски, связанные с проектом?
- Какие из диаграмм удобнее всего превратить в инструмент контроля над процессом (и прогрессом) разработки системы?

И еще одно - *никогда не выбрасывайте даже "забракованные" диаграммы*: они могут в дальнейшем оказаться полезными при анализе направления вашей мысли, поиске ошибок проектирования, да и просто для экспериментов по незначительному изменению системы.

ООП и последовательность построения диаграмм

Диаграммы, как уже говорилось выше, можно и нужно строить в некоторой логической последовательности. Но как выработать эту последовательность, если у вас нет опыта моделирования? Как научиться этому? Вот несколько простых приемов, которые помогут вам (или вашей команде) выработать свой стиль проектирования.

В *UML*-проектировании, как и при создании любых других моделей, важно уметь **абстрагироваться** от несущественных свойств системы. В этом плане очень полезными могут оказаться *коллективные упражнения на выявление и анализ прецедентов*. Они помогут отработать навыки выявления четких абстракций.

Неплохой способ начать - *моделирование базовых абстракций или поведения одной из уже имеющихся у вас систем*.

Стройте *модели предметной области задачи в виде диаграммы классов*! Это хороший способ понять, как визуализировать множества взаимосвязанных абстракций. Таким же образом стройте модели статической части задач.

Моделируйте *динамическую часть задачи с помощью простых диаграмм последовательностей и кооперации*. Хорошо начать с модели взаимодействия пользователя с системой - так вы сможете легко выделить наиболее важные прецеденты.

ООП и последовательность построения диаграмм

Не забываем, что мы говорим, прежде всего, именно об объектно-ориентированных системах. Поэтому можно предложить такую последовательность построения диаграмм:

- *диаграмма прецедентов,*
- *диаграмма классов,*
- *диаграмма объектов,*
- *диаграмма последовательностей,*
- *диаграмма кооперации,*
- *диаграмма состояний,*
- *диаграмма активности,*
- *диаграмма развертывания.*

Конечно, это не единственная возможная последовательность. Возможно, вам будет удобнее начать с *диаграммы классов*. А может, вам не нужны *диаграммы объектов*, а *диаграммы последовательностей* вы предпочитаете *диаграммам кооперации*. Это лишь один из путей, постепенно вы выработаете свой персональный стиль проектирования и свою последовательность!

И напоследок еще несколько советов относительно использования *UML*

- Хорошее и полезное упражнение - строить модели классов и отношений между ними для уже написанного вами кода на C++ или *Java*.
- Применяйте *UML* для того, чтобы прояснить неявные детали реализации существующей системы или использованные в ней "хитрые механизмы программирования".
- Стройте *UML*-модели, прежде чем начать новый проект. Только когда будете абсолютно удовлетворены полученным результатом, начинайте использовать их как основу для кодирования.
- Обратите особое внимание на средства *UML* для моделирования компонентов, параллельности, распределенности, паттернов проектирования.
- *UML* содержит некоторые средства расширения. Подумайте, как можно приспособить язык к предметной области вашей задачи. И не слишком увлекайтесь обилием средств *UML*: если вы в каждой диаграмме будете использовать абсолютно все средства *UML*, прочесть созданную вами модель смогут лишь самые опытные пользователи.
- Кроме прочего, важным моментом здесь является выбор пакета *UML*-моделирования (CASE-средства), что тоже может повлиять на ваш индивидуальный стиль проектирования.

Выводы

- Диаграммы разных видов позволяют взглянуть на систему с разных точек зрения.
- UML содержит диаграммы трех типов - для моделирования статической структуры, поведенческих аспектов и подробностей реализации приложения.

Спасибо за внимание!

Ищенко Алексей Петрович